Федеральное государственное автономное
образовательное учреждение высшего образования
«Санкт-Петербургский национальный исследовательский университет
информационных технологий, механики и оптики»
Факультет Программной Инженерии и Компьютерных Технологий

Лабораторная работа №4

«Анализ трафика компьютерных сетей с помощью утилиты **Wireshark**»
по дисциплине

«Компьютерные сети»

Выполнил:
Носов Георгий Иванович
P3332

Преподаватель:
Алиев Тауфик Измайлович
Отчёт принят «__»____2026 г.
Оценка: _____

Санкт-Петербург
2026

Содержание

1. Цель работы 2

Вариант 2

2. Этапы и порядок выполнения работы 3

3. Ход работы 6

Задание 4.1. Анализ трафика утилиты ping 6

Задание 4.2. Анализ трафика утилиты tracert (traceroute) 9

Задание 4.3. Анализ HTTP-трафика 12

Задание 4.5. Анализ ARP-трафика 14

Задание 4.6. Анализ трафика утилиты nslookup 16

4. Структура наблюдаемых пакетов 19

5. Выводы по работе: 19

1. Цель работы

Целью данной лабораторной работы является изучение структуры протокольных блоков данных, путём анализа реального трафика на компьютере студента с помощью бесплатно распространяемой утилиты Wireshark.

В процессе выполнения домашнего задания выполняются наблюдения за передаваемым трафиком с компьютера пользователя в Интернет и в обратном направлении. Применение специализированной утилиты Wireshark позволяет наблюдать структуру передаваемых кадров, пакетов и сегментов данных различных сетевых протоколов. При выполнении УИР рекомендуется выполнить анализ последовательности команд и определить назначение служебных данных, используемых для организации обмена данными в протоколах: ARP, DNS, FTP, HTTP, DHCP.

Вариант

Для выполнения лабораторной работы будут представлены пункты 4.1, 4.2, 4.3, 4.5, 4.6.

В качестве сайта по варианту выбран сайт **nginx.org**: Носов Георгий Иванович - **NGI**.

Во время захвата сайт nginx.org разрешался в IPv4-адрес: **3.125.197.172**.

2. Этапы и порядок выполнения работы

4.1. Анализ трафика утилиты ping

Необходимо отследить и проанализировать трафик, создаваемый утилитой **ping**, запустив её следующим образом из командной строки:

«ping -l размер_пакета адрес_сайта_по_варианту».

Например, «ping -l 2000 wireshark.org» (без кавычек).

В качестве «размера_пакета» необходимо поочерёдно использовать различные значения от 100 до 10000, самостоятельно выбрав шаг изменения. По результатам анализа собранной трассы, необходимо ответить на следующие вопросы и выполнить указанные задания.

1. Имеет ли место фрагментация исходного пакета, какое поле на это указывает?
2. Какая информация указывает, является ли фрагмент пакета последним или промежуточным?
3. Чему равно количество фрагментов при передаче ping-пакетов?
4. Построить график, в котором на оси абсцисс находится размер_пакета, а по оси ординат – количество фрагментов, на которое был разделён каждый ping-пакет.
5. Как изменить поле TTL с помощью утилиты ping?
6. Что содержится в поле данных ping-пакета?

4.2. Анализ трафика утилиты tracert (traceroute)

Необходимо отследить и проанализировать трафик, создаваемый утилитой **tracert** (или **traceroute** в Linux), запустив её следующим образом из командной строки:

«tracert -d адрес_сайта_по_варианту»

Например, tracert wireshark.org.

По результатам анализа собранной трассы, ответьте на следующие вопросы.

1. Сколько байт содержится в заголовке IP? Сколько байт содержится в поле данных?
2. Как и почему изменяется поле TTL в следующих друг за другом ICMP-пакетах tracert? Для ответа на этот вопрос нужно проследить изменение TTL при передаче по маршруту, состоящему из более чем двух хопов.
3. Чем отличаются ICMP-пакеты, генерируемые утилитой tracert, от ICMP-пакетов, генерируемых утилитой ping (см. предыдущее задание).
4. Чем отличаются полученные пакеты «ICMP reply» от «ICMP error» и зачем нужны оба этих типа ответов?
5. Что изменится в работе tracert, если убрать ключ «-d»? Какой дополнительный трафик при этом будет генерироваться?

4.3. Анализ HTTP-трафика

Необходимо отследить и проанализировать **HTTP-трафик**, создаваемый браузером при посещении Интернет-сайта, заданного по варианту. В списке захваченных пакетов необходимо проанализировать следующую пару HTTP-сообщений (запрос-ответ):

- GET-сообщение от клиента (браузера);
- ответ сервера.

Для этого в поле с детальной информацией о пакете нужно развернуть строку «HTTP». Затем необходимо обновить страницу в браузере так, чтобы вместо «HTTP GET» был сгенерирован «HTTP CONDITIONAL GET» (так называемый «условный GET»). Условные запросы GET содержат поля If-Modified-Since, If-Match, If-Range и подобные, которые позволяют при повторном запросе не передавать редко изменяемые данные. В ответ на условный GET тело запрашиваемого ресурса передается только в том случае, если этот ресурс изменялся после даты «If-Modified-Since». Если ресурс не изменялся, сервер вернет код статуса «304 Not Modified».

По результатам анализа собранной трассы покажите, каким образом протокол HTTP передавал содержимое страницы при первичном посещении страницы и при вторичном запросе-обновлении от браузера (т.е. при различных видах GET-запросов).

4.5. Анализ ARP-трафика

Необходимо отследить и проанализировать трафик протокола **ARP**, сгенерированный в результате выполнения следующих действий:

- очистить ARP-таблицу командой «netsh interface ip delete arpcache» (проверить очистилась ли таблица можно с помощью команды команды «arp -a», выводящей таблицу на экран);
- очистить кэш браузера;
- зайти на Интернет-сайт, заданный по варианту.

По результатам анализа собранной трассы, ответьте на следующие вопросы.

1. Какие MAC-адреса присутствуют в захваченных пакетах ARP- протокола? Что означают эти адреса? Какие устройства они идентифицируют?
2. Какие MAC-адреса присутствуют в захваченных HTTP-пакетах и что означают эти адреса? Что означают эти адреса? Какие устройства они идентифицируют?
3. Для чего ARP-запрос содержит IP-адрес источника?

4.6. Анализ трафика утилиты nslookup

Необходимо отследить и проанализировать трафик протокола **DNS**,

сгенерированный в результате выполнения следующих действий:

1. Настроить Wireshark-фильтр: «ip.addr == ваш_IP_адрес».

2. Запустить в командной строке команду «nslookup адрес_сайта_по_варианту».
3. Дождаться отправки трёх DNS-запросов и трёх DNS-ответов (в работе нужно использовать только последние из них, т.к. первые два набора запросов/ответов специфичны для nslookup и не генерируются другими сетевыми приложениями).
4. Повторить предыдущие два шага, используя команду: «nslookup -type=NS имя_сайта_по_варианту».

По результатам анализа собранной трассы, ответьте на следующие вопросы.

1. Чем различается трасса трафика в п.2 и п.4, указанных выше?
2. Что содержится в поле «Answers» DNS-ответа?
3. Каковы имена серверов, возвращающих авторитативный (authoritative) отклик?

3. Ход работы

Задание 4.1. Анализ трафика утилиты ping

Для анализа был выполнен захват ICMP-трафика при отправке ping-пакетов на адрес nginx.org с различным размером поля данных. На macOS использовалась опция -s, задающая размер ICMP payload.

ping -c 1 -s 100 nginx.org
ping -c 1 -s 1000 nginx.org
ping -c 1 -s 1500 nginx.org
ping -c 1 -s 2000 nginx.org
ping -c 1 -s 3000 nginx.org
ping -c 1 -s 5000 nginx.org
ping -c 1 -s 8000 nginx.org

Для размера 10000 байт macOS вернула ошибку sendto: Message too long, поэтому максимальный реально отправленный размер в захвате составил 8000 байт.

Фильтр Wireshark:

ip.addr == 3.125.197.172

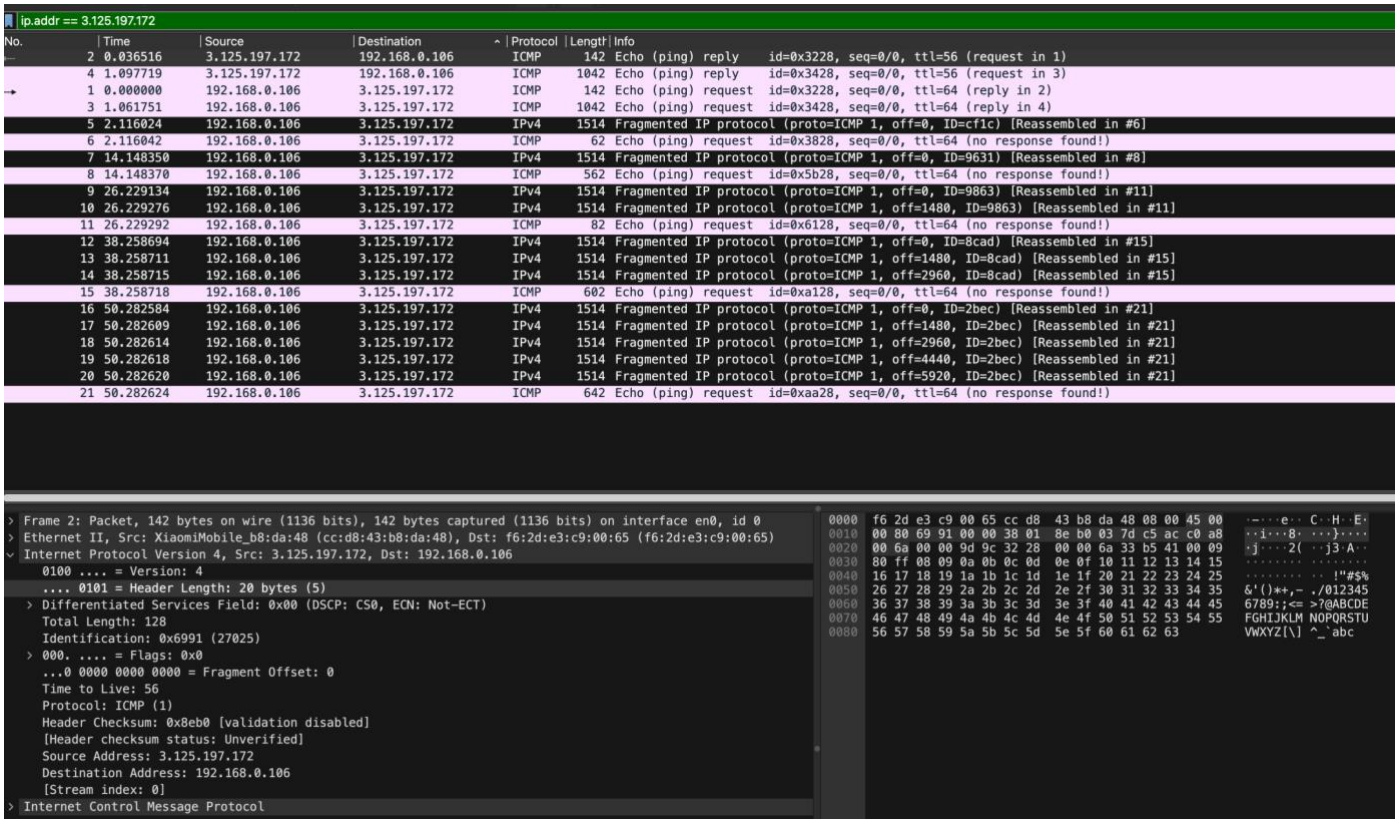


Рисунок 1 - ICMP Echo Request без фрагментации.

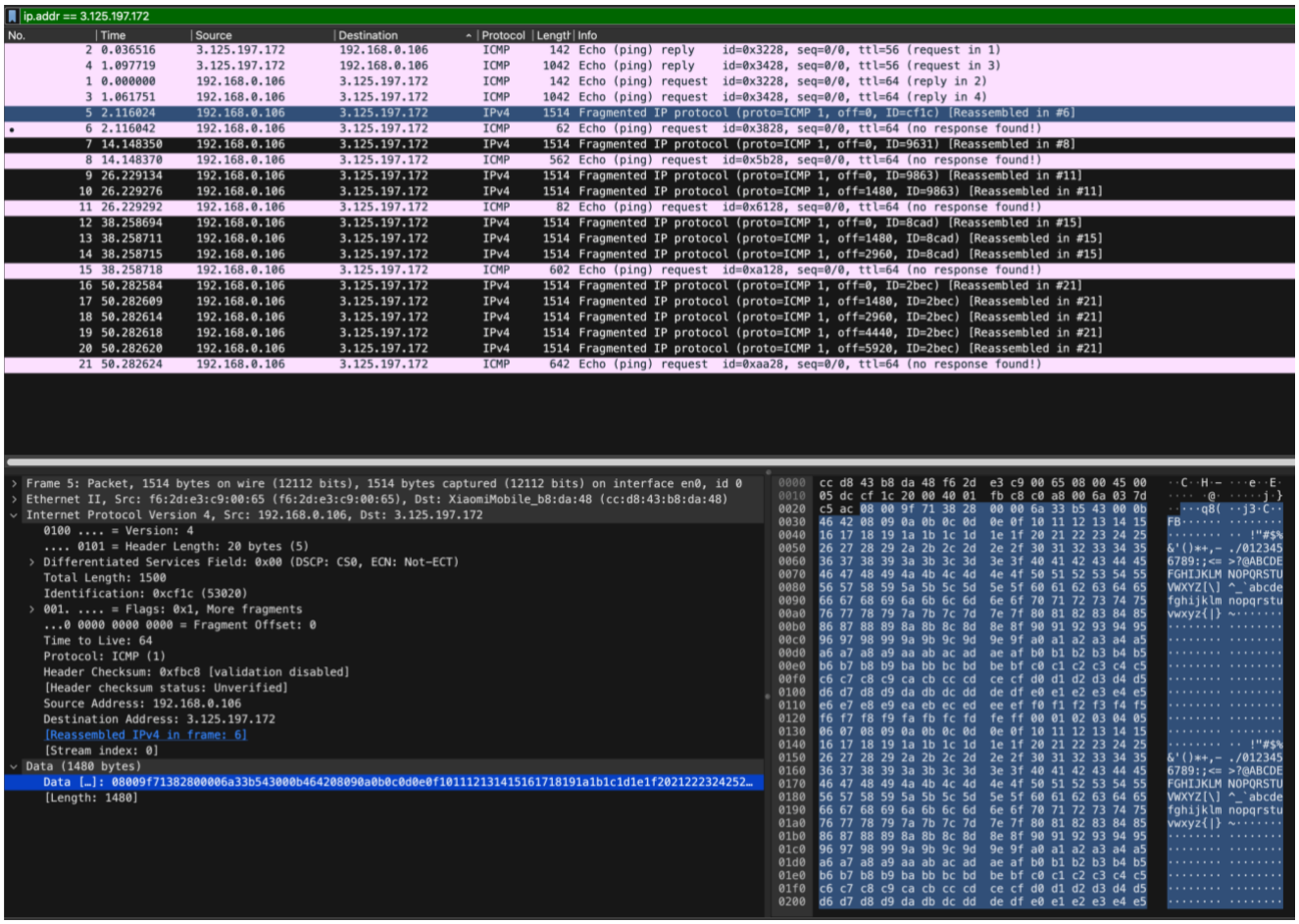


Рисунок 2 - Первый фрагмент ping-пакета.

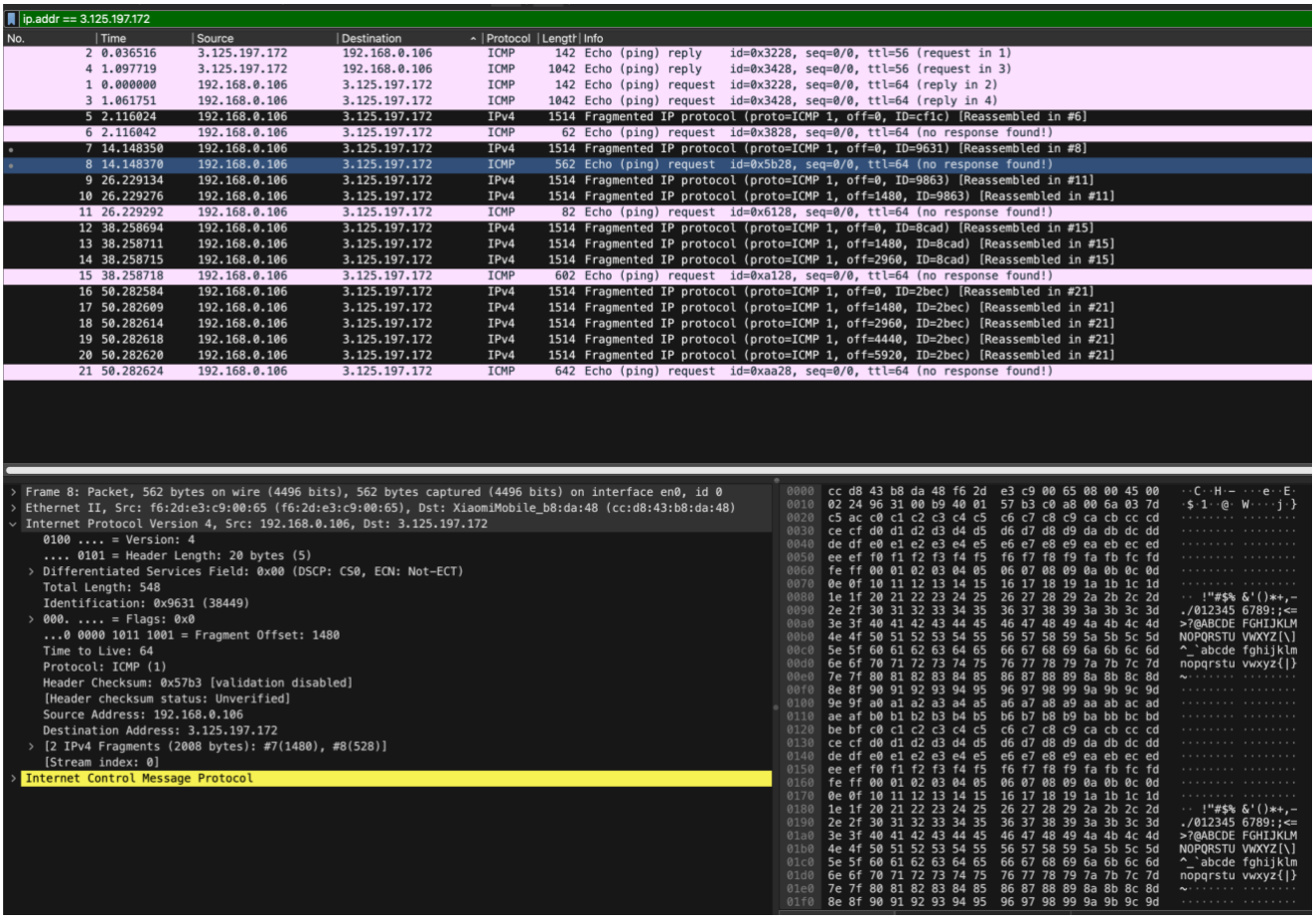


Рисунок 3 - Последний фрагмент ping-пакета.

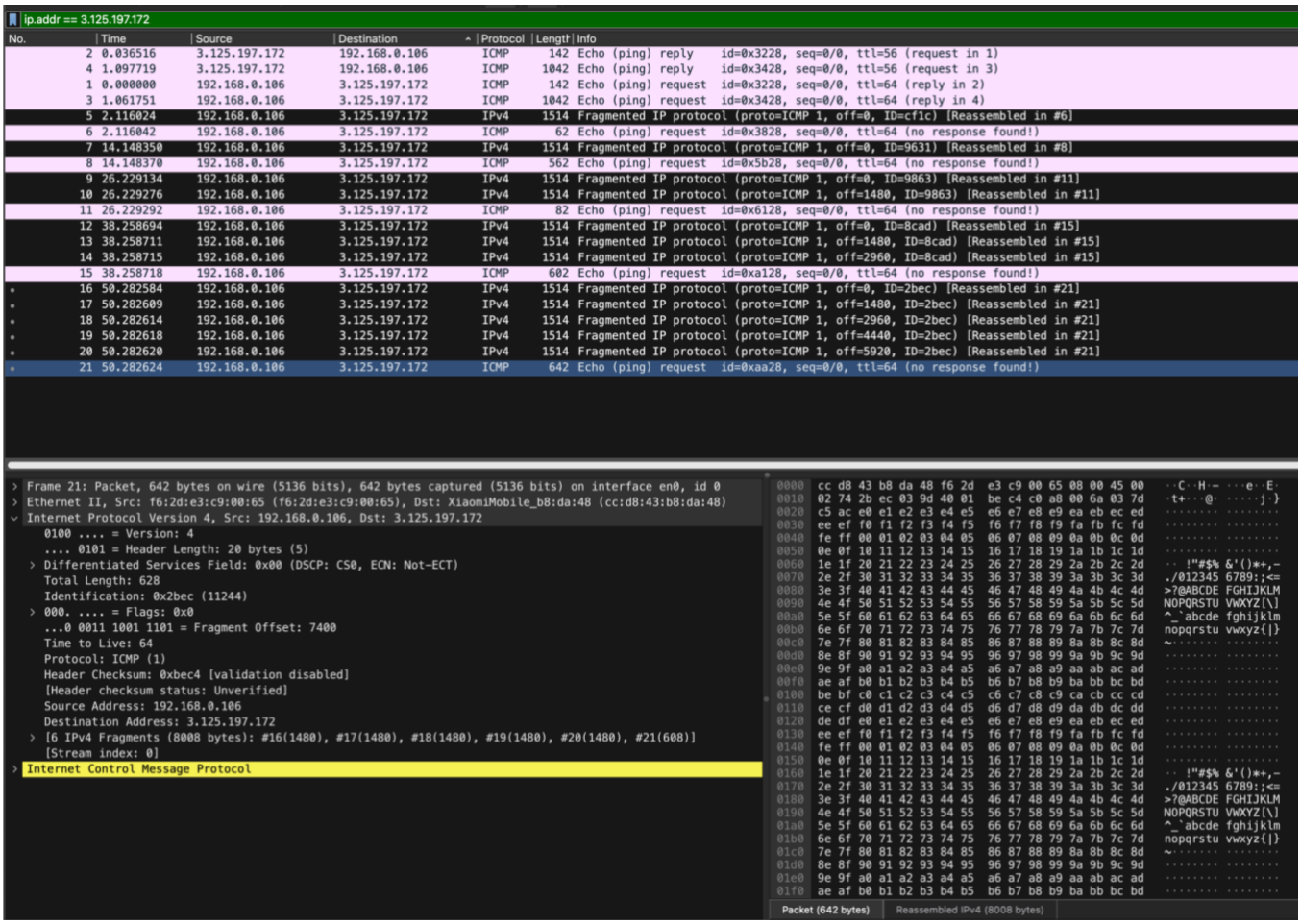


Рисунок 4 - Фрагментация ping-пакета большого размера.

Структура ICMP-пакета

1. Канальный уровень: Ethernet II. Заголовок содержит MAC-адрес источника, MAC-адрес назначения и тип вложенного протокола.
2. Сетевой уровень: IPv4. Заголовок содержит Version, Header Length, Total Length, Identification, Flags, Fragment Offset, TTL, Protocol, Header Checksum, Source Address и Destination Address.
3. Протокол ICMP. Заголовок содержит Type, Code, Checksum, Identifier и Sequence Number.
4. Поле данных ICMP содержит тестовую последовательность байтов, отправляемую утилитой ping.

Ответы на вопросы

1. Имеет ли место фрагментация исходного пакета, какое поле на это указывает?

При увеличении размера ICMP payload сверх значения, которое может быть передано одним пакетом в сети с MTU 1500, исходный пакет фрагментируется. На фрагментацию указывают поле Flags, установленный флаг More fragments, ненулевое поле Fragment Offset, а также одинаковое значение Identification у фрагментов одного исходного пакета.

2. Какая информация указывает, является ли фрагмент последним или промежуточным?

Если флаг More fragments установлен, фрагмент является промежуточным. Если флаг More fragments не установлен, но Fragment Offset больше нуля, это последний фрагмент исходного IP-пакета.

3. Чему равно количество фрагментов при передаче ping-пакетов?

Учитывая, что один фрагмент по MTU равен примерно 1480 байт, кол-во фрагментов будет равно размеру пакета / 1480 и округлить до верхнего целого числа.

4. График зависимости количества фрагментов от размера ping-пакета.

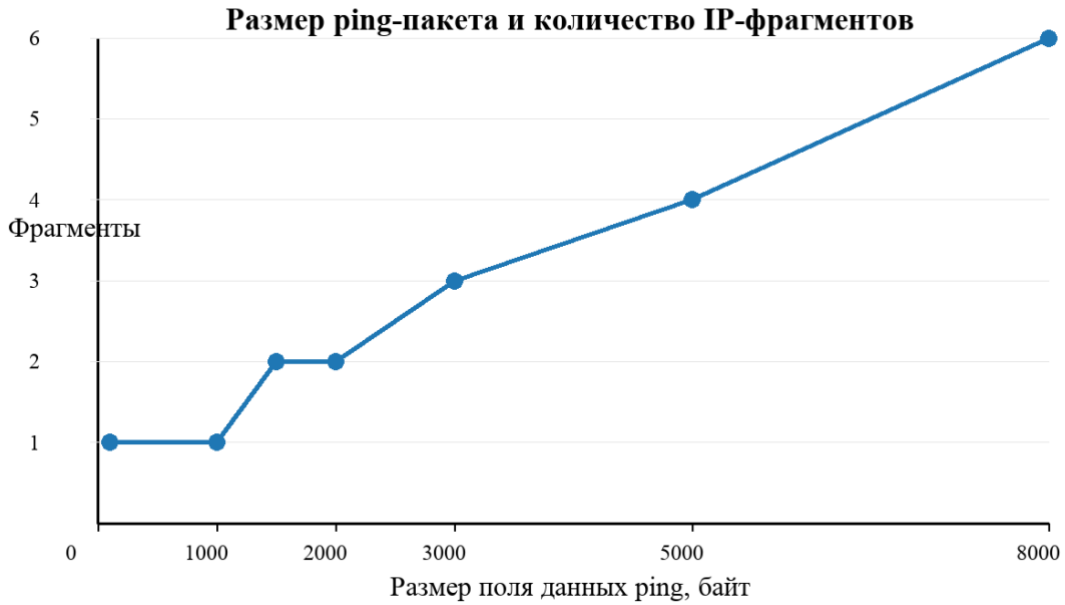


Рисунок 5 - Зависимость количества фрагментов от размера поля данных ping.

5. Как изменить поле TTL с помощью утилиты ping?

На macOS значение TTL для исходящих ping-пакетов изменяется ключом -m. Например:

ping -c 3 -m 5 nginx.org

В Windows для аналогичной цели используется ключ -i, например **ping -i 5 nginx.org**.

6. Что содержится в поле данных ping-пакета?

В поле данных находится тестовая последовательность байтов, которую утилита ping передаёт в составе Echo Request.

Задание 4.2. Анализ трафика утилиты tracert (traceroute)

Для macOS использовалась утилита traceroute с ключом -I, чтобы отправлять ICMP Echo Request, и ключом -n, чтобы отключить DNS-разрешение промежуточных узлов.

traceroute -I -n -q 1 -m 12 -w 2 nginx.org

Фильтр Wireshark: icmp

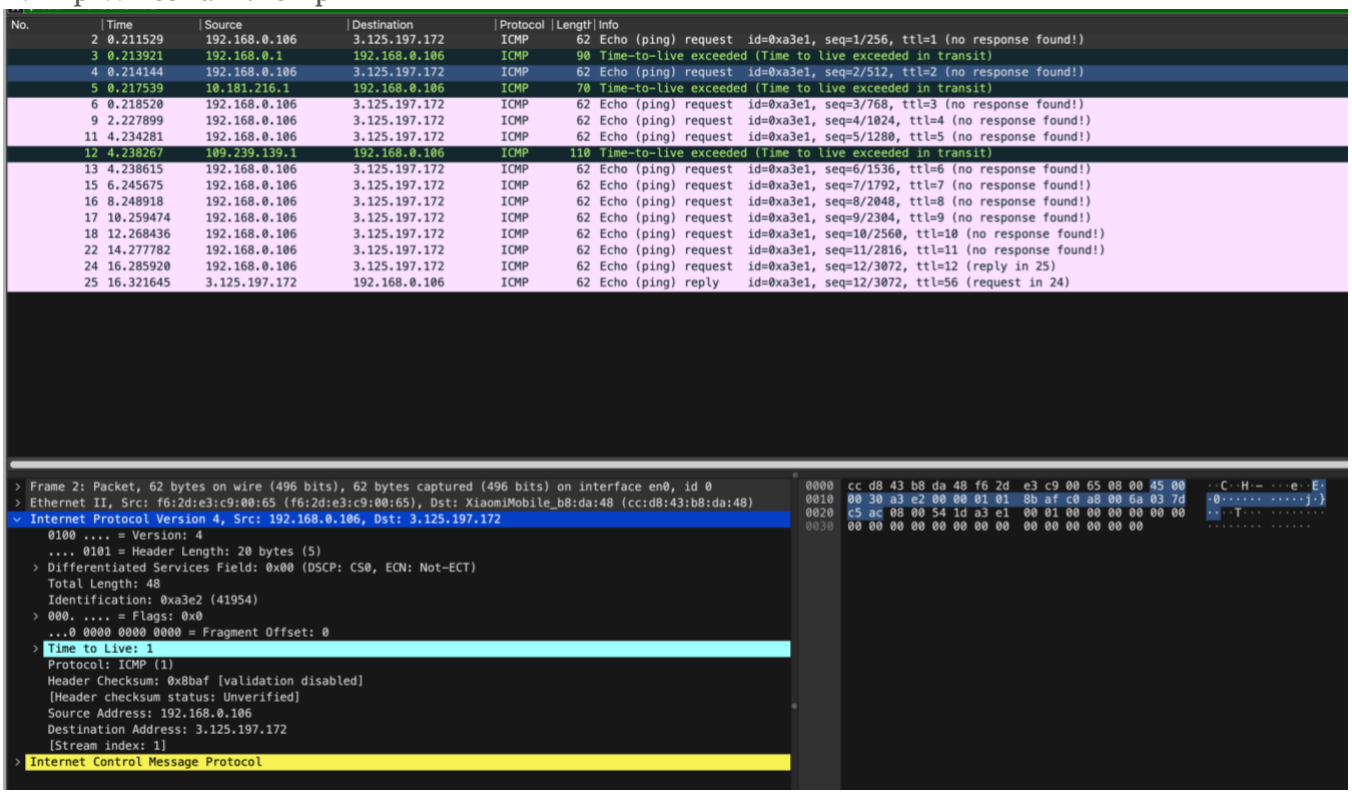


Рисунок 6 - ICMP Echo Request с TTL = 1.

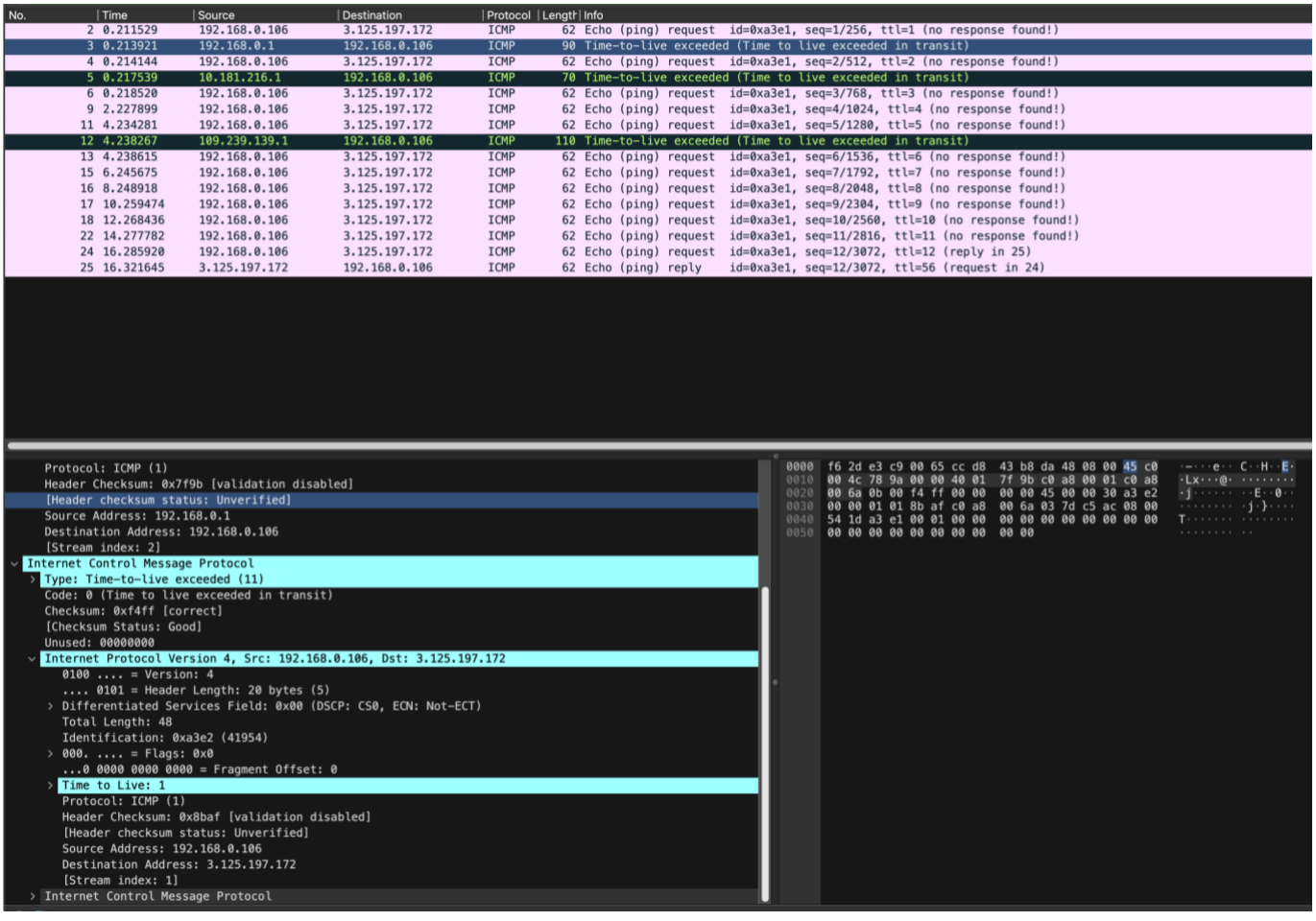


Рисунок 7 - Ответ ICMP Time-to-live exceeded от промежуточного маршрутизатора.

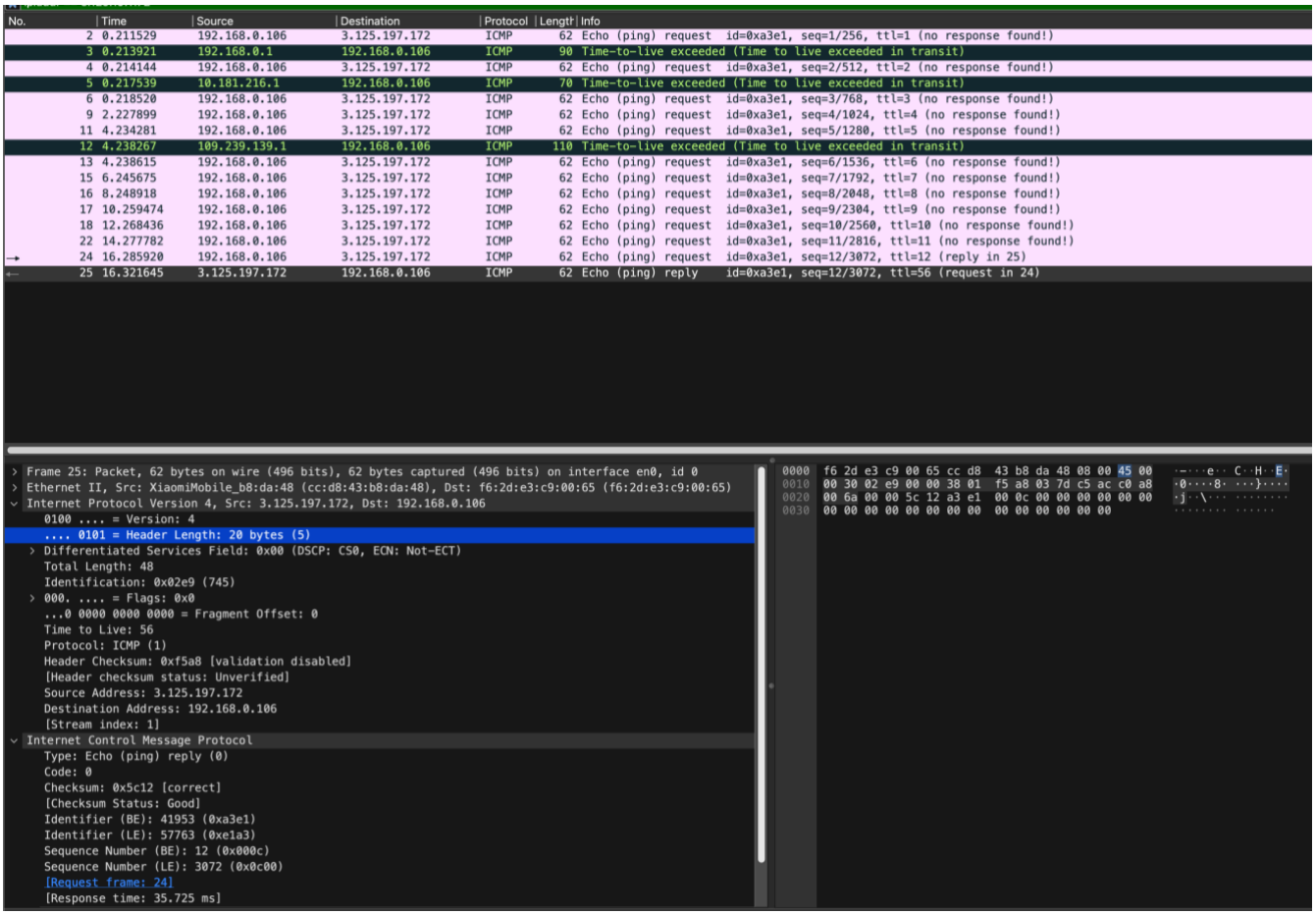


Рисунок 8 - Ответ ICMP Echo Reply от конечного узла.

Ответы на вопросы

1. Сколько байт содержится в заголовке IP? Сколько байт содержится в поле данных?

В наблюдаемых пакетах длина IP-заголовка составляет 20 байт. Поле данных пакета содержит ICMP сообщение: заголовок 8 байт и поле данных ICMP.

2. Как и почему изменяется поле TTL в следующих друг за другом ICMP-пакетах traceroute?

Traceroute отправляет последовательность ICMP Echo Request с возрастающим TTL: 1, 2, 3 и далее. Каждый маршрутизатор на пути уменьшает TTL на единицу. Когда TTL становится равным нулю, маршрутизатор отбрасывает пакет и отправляет ICMP Time Exceeded. Благодаря этому утилита определяет промежуточные узлы маршрута.

3. Чем отличаются ICMP-пакеты traceroute от ICMP-пакетов ping?

Ping отправляет ICMP Echo Request с обычным TTL и проверяет достижимость узла. Traceroute также отправляет ICMP Echo Request, но специально меняет TTL, чтобы получать ICMP Time Exceeded от промежуточных маршрутизаторов и построить маршрут.

4. Чем отличаются ICMP reply от ICMP error и зачем нужны оба?

ICMP Echo Reply (type 0) приходит от конечного узла и подтверждает, что адресат достигнут. ICMP Time Exceeded (type 11) является сообщением об ошибке от

промежуточного маршрутизатора, у которого истёк TTL пакета. В traceroute оба типа сообщений нужны: error пакеты показывают промежуточные хопы, а reply показывает конец маршрута.

5. Что изменится в работе tracert, если убрать ключ «-d» (-n)? Какой дополнительный трафик при этом будет генерироваться?

Ключ -n (для MacOS) запрещает обратное DNS-разрешение IP-адресов промежуточных узлов. Если убрать этот ключ, traceroute будет пытаться получить доменные имена для IP-адресов маршрутизаторов, что приведёт к появлению дополнительного DNS-трафика.

Задание 4.3. Анализ HTTP-трафика

Для анализа HTTP был использован адрес http://nginx.org/. Сначала был выполнен обычный GET-запрос, затем условный GET-запрос с заголовками If-None-Match и If-Modified-Since.
Фильтр Wireshark: http

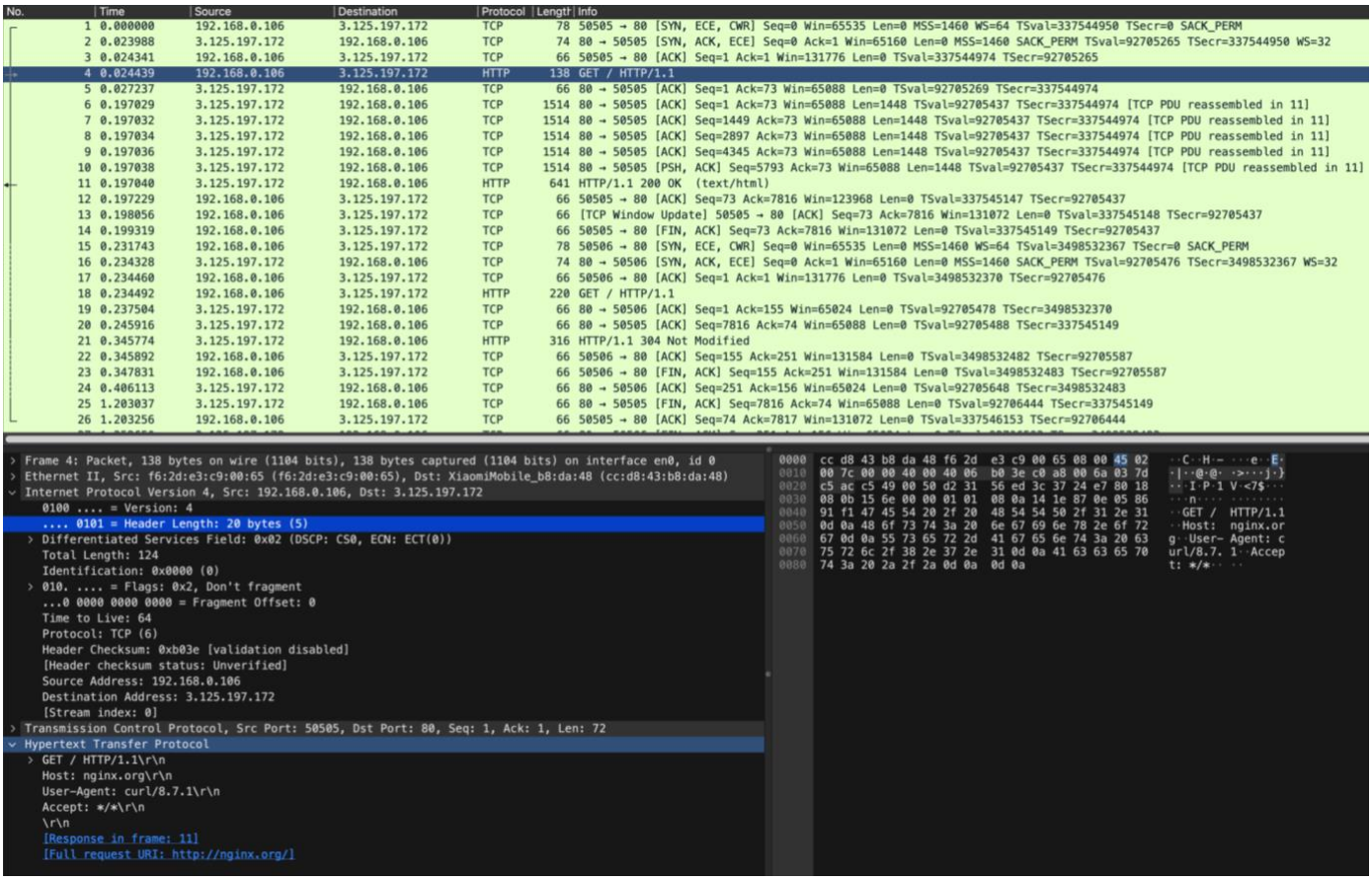


Рисунок 9 - Первый HTTP GET-запрос.

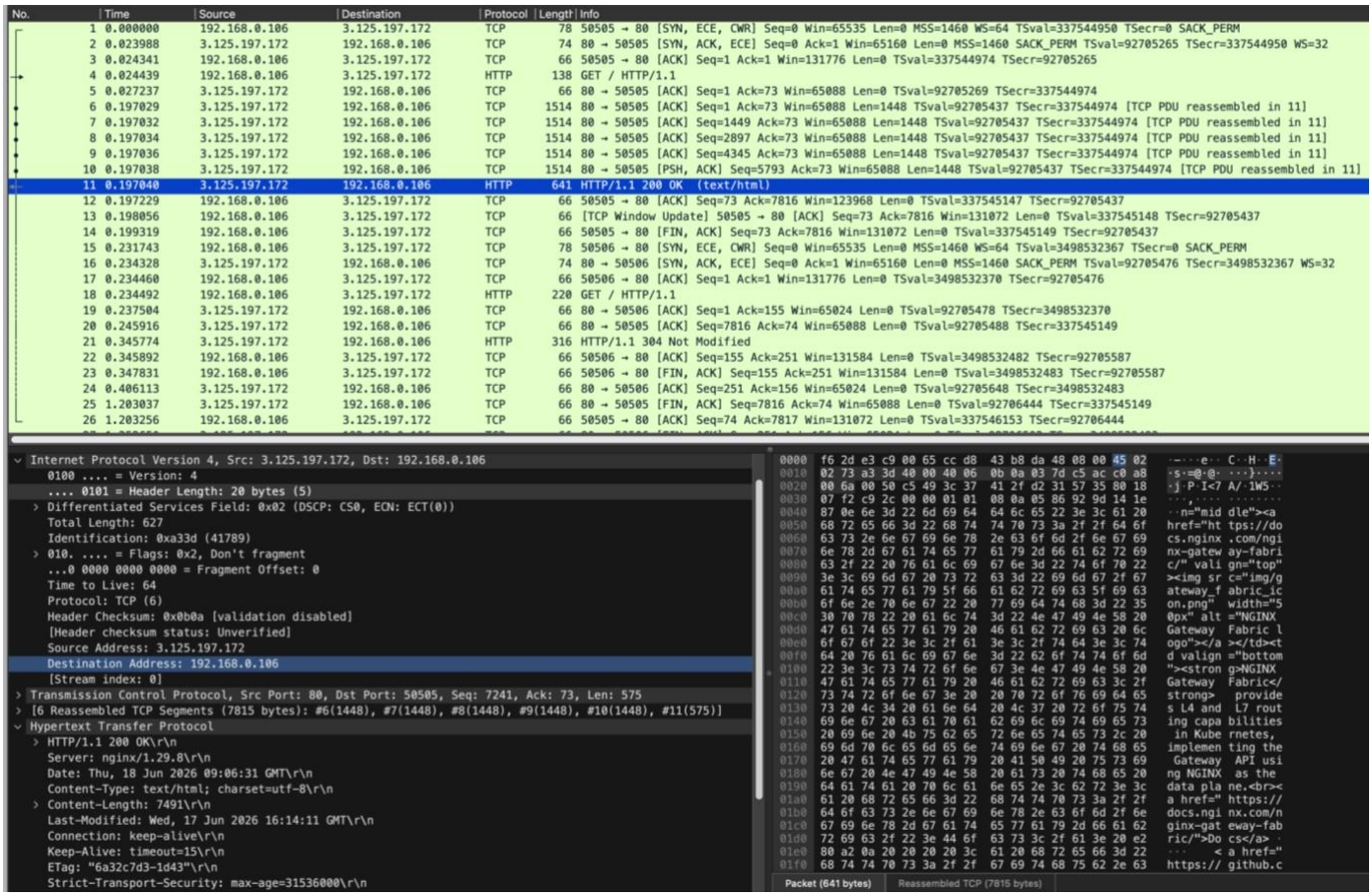


Рисунок 10 - Ответ сервера HTTP/1.1 200 OK.

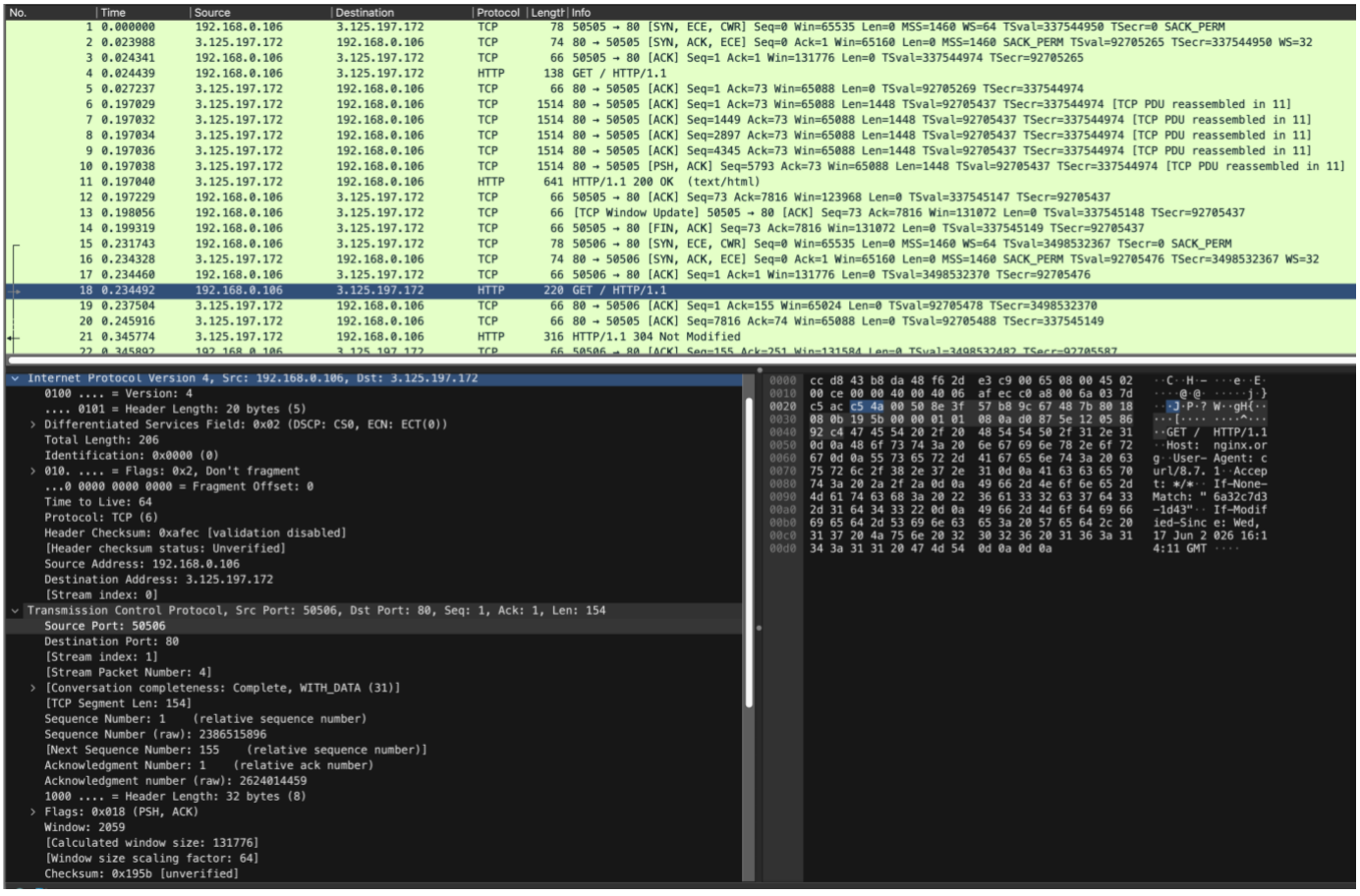


Рисунок 11 - Условный HTTP GET-запрос.

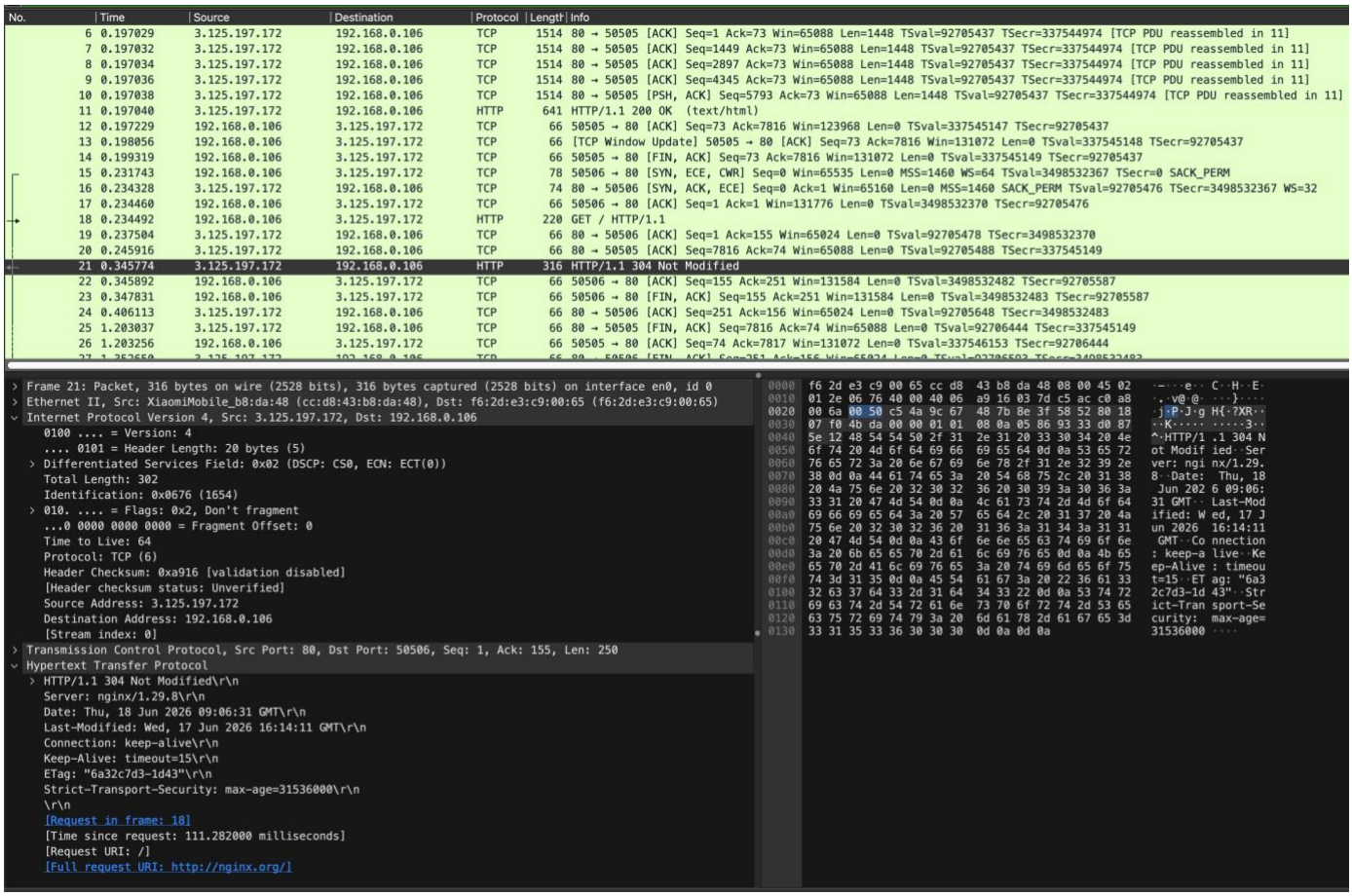


Рисунок 12 - Ответ сервера HTTP/1.1 304 Not Modified.

Ответ на вопрос

При первичном обращении клиент отправил GET / HTTP/1.1, а сервер вернул HTTP/1.1 200 OK и передал тело HTML-страницы. В ответе присутствовали заголовки Last-Modified и ETag: "6a32c7d3-1d43".

При повторном обращении клиент отправил условный GET с заголовками If-None-Match: "6a32c7d3-1d43" и If-Modified-Since. Так как ресурс не изменился, сервер вернул HTTP/1.1 304 Not Modified. В этом случае тело страницы повторно не передаётся, что уменьшает объем сетевого трафика.

Задание 4.5. Анализ ARP-трафика

Для генерации ARP-трафика была очищена ARP-запись для шлюза, после чего было выполнено обращение к сайту nginx.org. На macOS для удаления записи ARP использовалась команда:

sudo arp -d 192.168.0.1

Фильтр Wireshark: arp

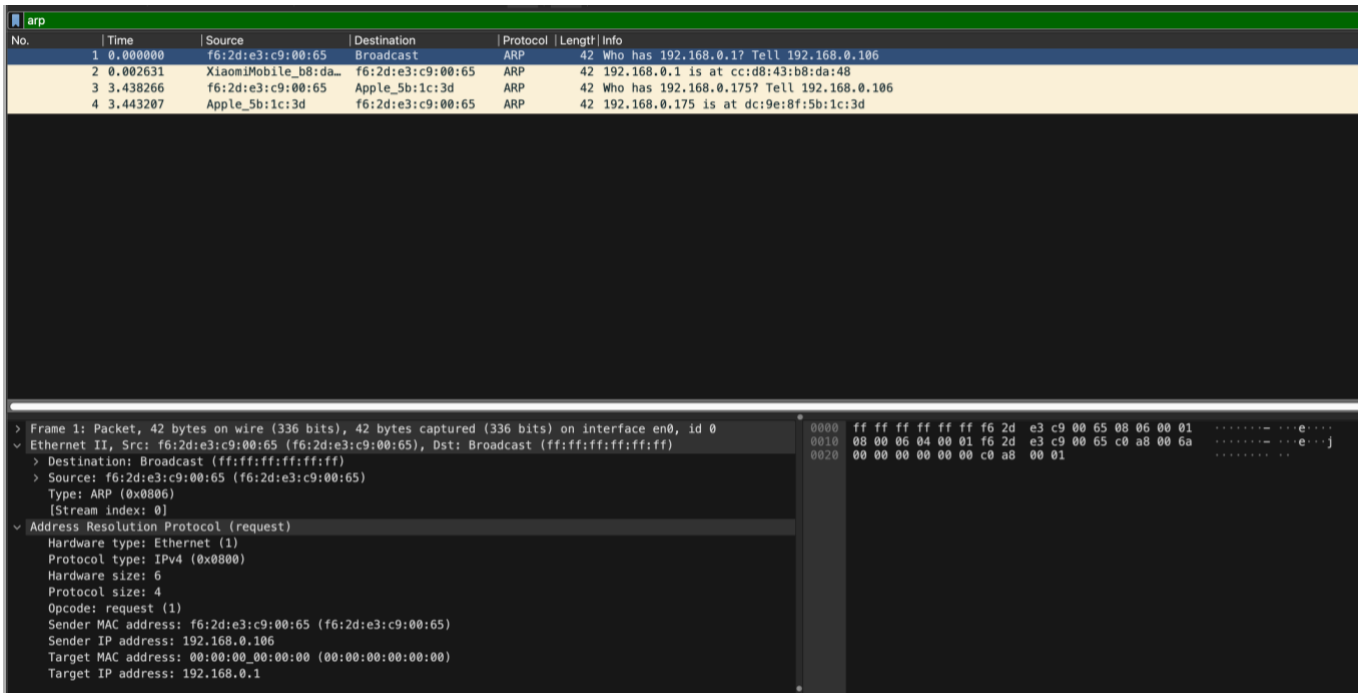


Рисунок 13 - ARP-запрос от компьютера к шлюзу.

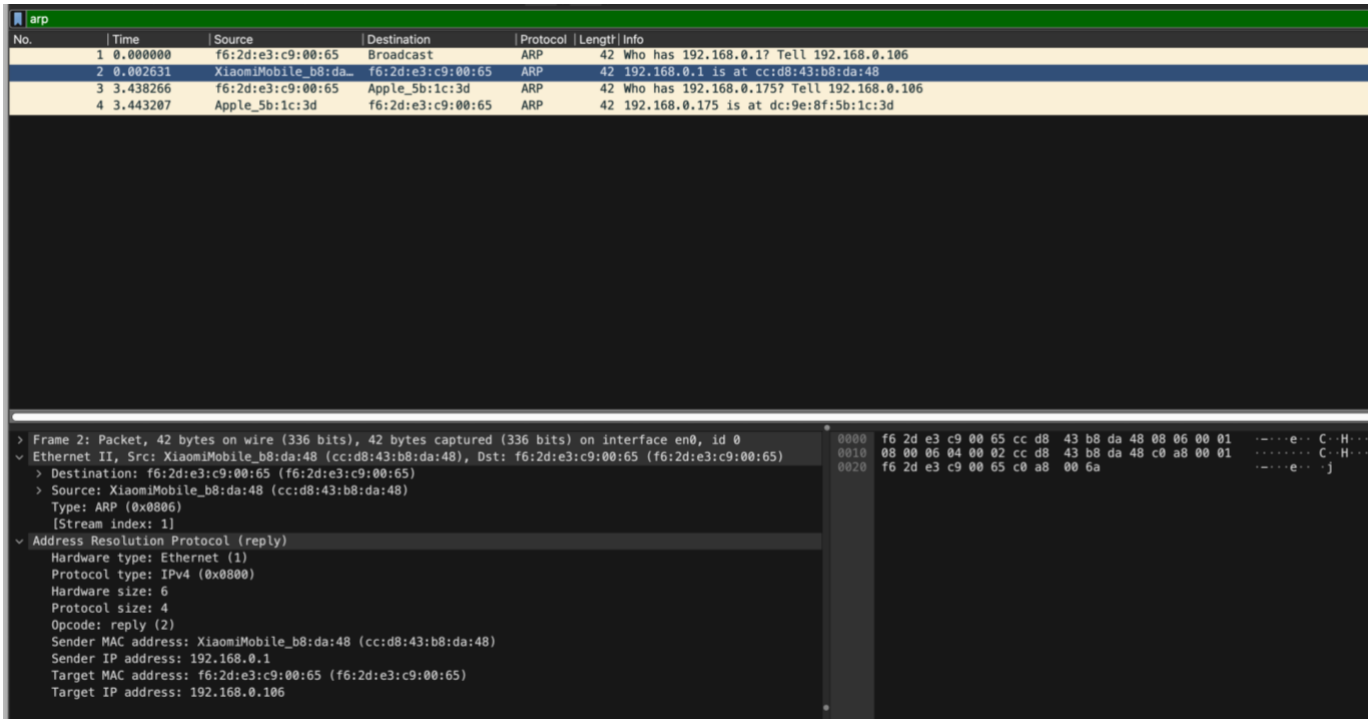


Рисунок 14 - ARP-ответ от шлюза.

Ответы на вопросы

1. Какие MAC-адреса присутствуют в захваченных пакетах ARP протокола? Что означают эти адреса? Какие устройства они идентифицируют?

В ARP-запросе наш компьютер 192.168.0.106 с MAC-адресом f6:2d:e3:c9:00:65 отправляет широковещательный кадр на ff:ff:ff:ff:ff:ff с вопросом, какой MAC-адрес соответствует IP-адресу 192.168.0.1. Target MAC в запросе равен 00:00:00:00:00:00, потому что искомый MAC ещё неизвестен.

В ARP-ответе шлюз 192.168.0.1 сообщает свой MAC-адрес cc:d8:43:b8:da:48 и отправляет ответ напрямую компьютеру с MAC-адресом f6:2d:e3:c9:00:65.

2. Какие MAC-адреса присутствуют в захваченных HTTP-пакетах и что означают эти адреса? Что означают эти адреса? Какие устройства они идентифицируют?

В HTTP-пакетах к удалённому сайту в Ethernet-заголовке используются MAC-адреса локального участка сети: MAC компьютера f6:2d:e3:c9:00:65 и MAC ближайшего шлюза (роутера) cc:d8:43:b8:da:48.

3. Для чего ARP-запрос содержит IP-адрес источника?

IP-адрес источника нужен, чтобы узел-получатель мог сформировать ARP-ответ и, при необходимости, обновить свою ARP-таблицу соответствием IP-адреса и MAC-адреса отправителя.

Задание 4.6. Анализ трафика утилиты nslookup

Дополнительно был рассмотрен трафик DNS, который создаёт утилита nslookup. Сначала был выполнен обычный запрос адреса сайта, затем запрос NS-записей домена.

nslookup nginx.org
nslookup -type=NS nginx.org

Фильтр Wireshark: dns && dns.qry.name == "nginx.org"

DNS-запросы отправлялись с компьютера 192.168.0.106 на локальный DNS-сервер 192.168.0.1 по UDP-порту 53. Ответы приходили от DNS-сервера обратно на динамический UDP-порт клиента.

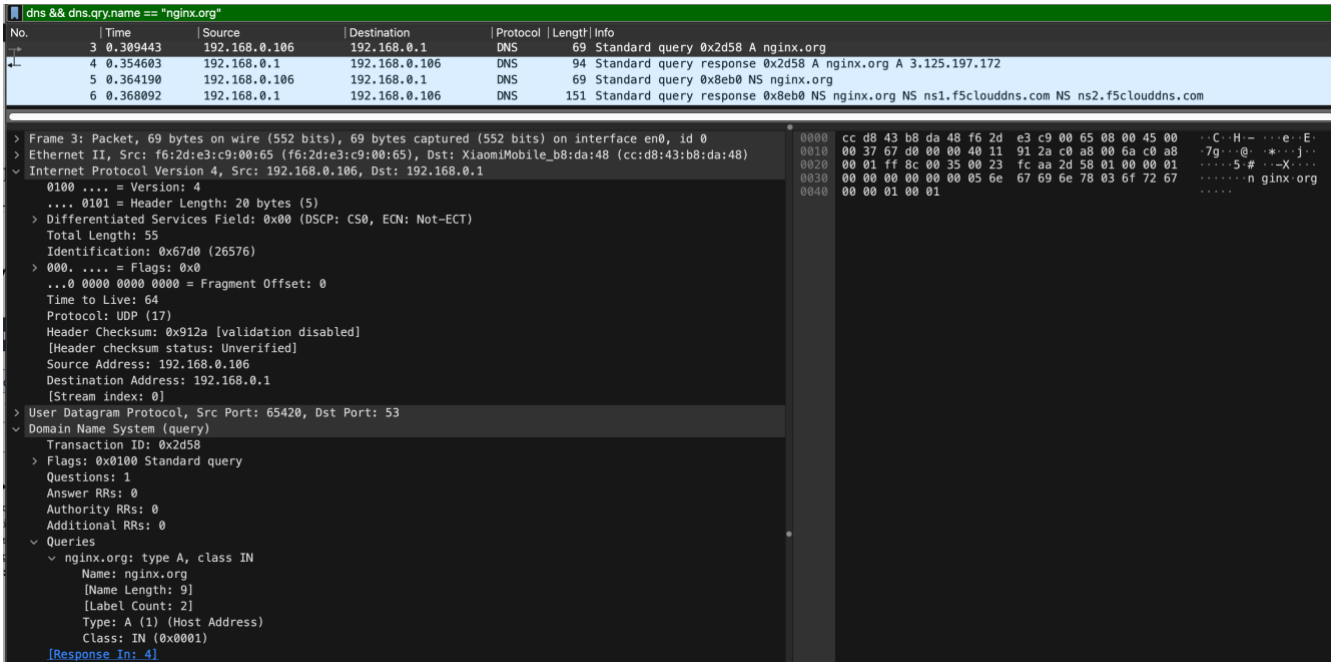


Рисунок 15 - DNS-запрос для записи типа A.

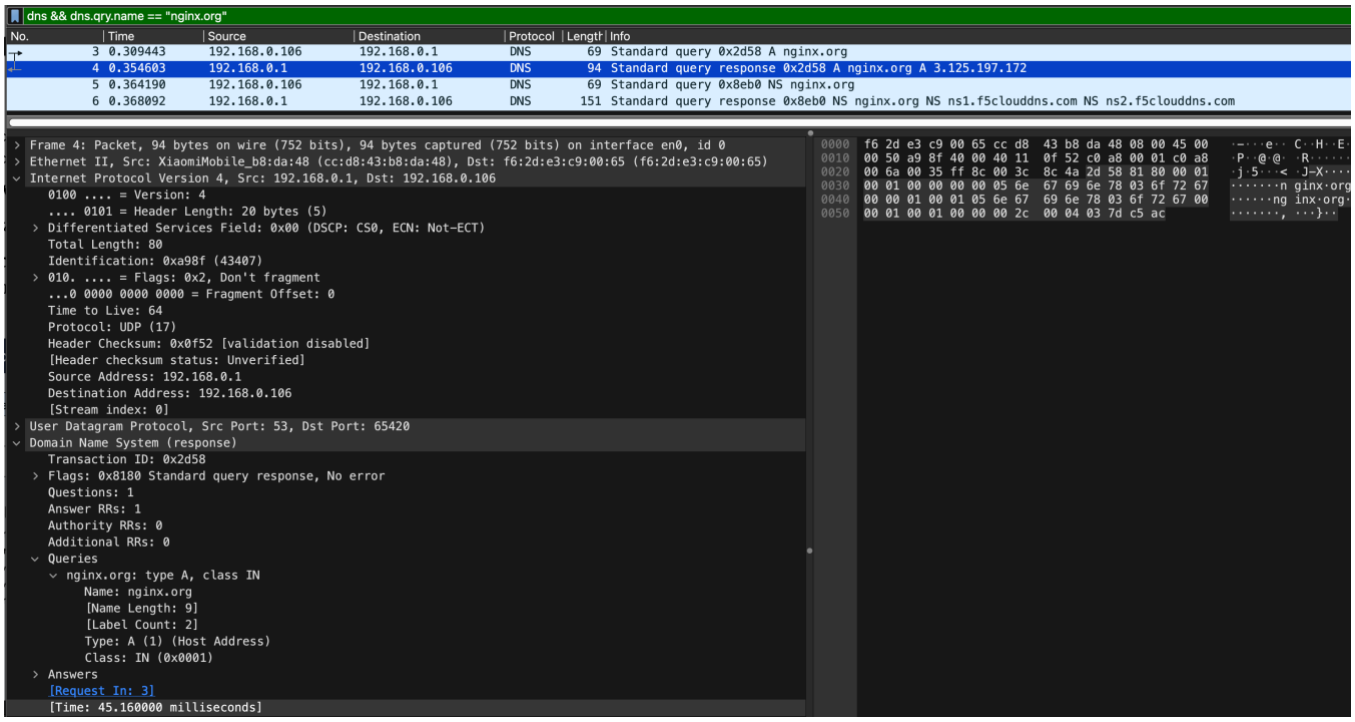


Рисунок 16 - DNS-ответ для записи типа A.

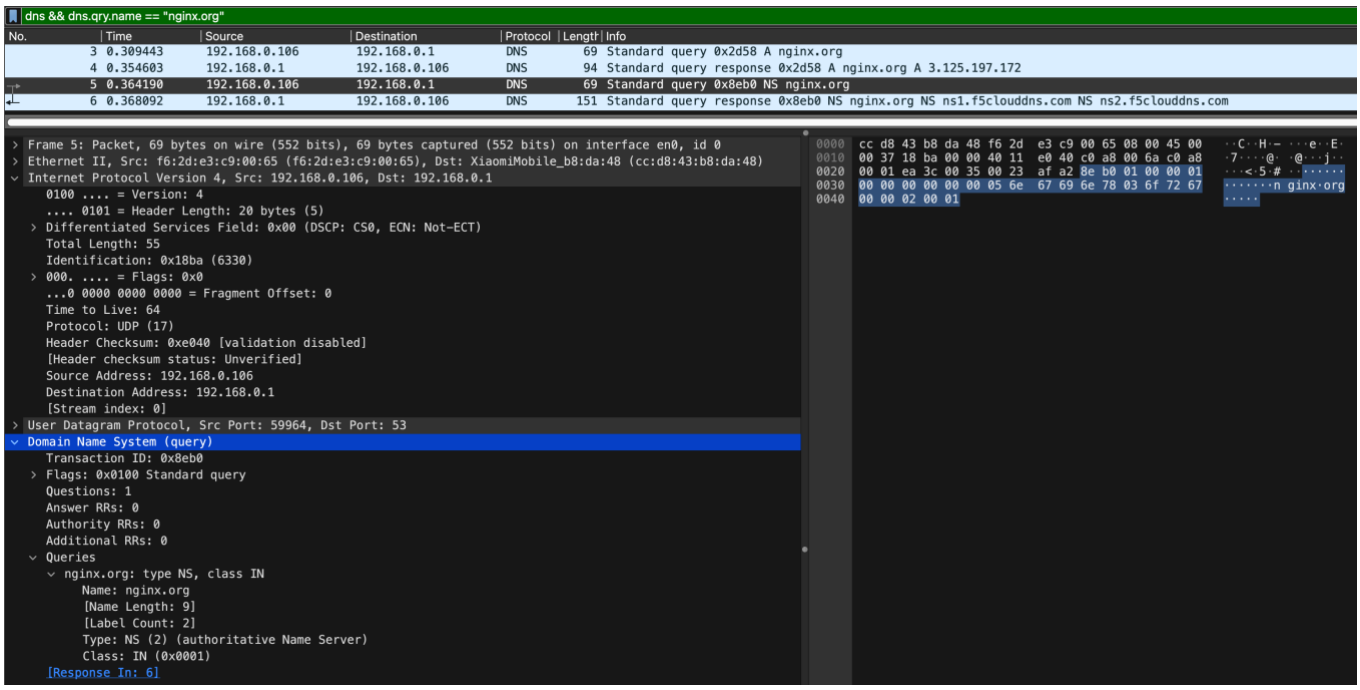


Рисунок 17 - DNS-запрос для записей типа NS.

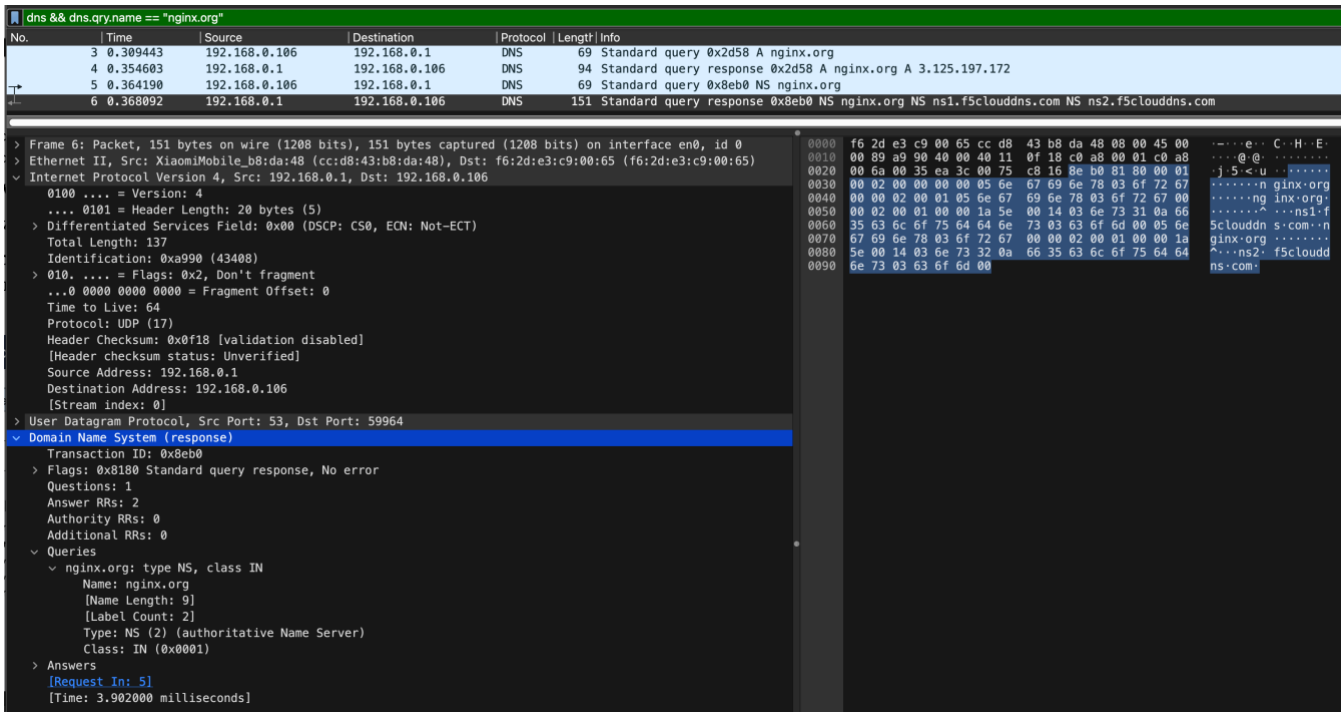


Рисунок 18 - DNS-ответ для записи типа NS.

Структура DNS-пакета

Канальный уровень: Ethernet II. Кадр передаётся между компьютером и локальным шлюзом/DNS-сервером.

Сетевой уровень: IPv4. IP-адрес источника и назначения соответствуют компьютеру и DNS-серверу.

Транспортный уровень: UDP. DNS обычно использует порт 53 на стороне сервера.

Прикладной уровень: DNS. Сообщение содержит Transaction ID, Flags, раздел Queries и для ответа раздел Answers.

Ответы на вопросы

1. Чем различается трасса трафика при обычном nslookup и при nslookup -type=NS?

В обычном nslookup утилита запрашивает адресную запись типа A для домена nginx.org. В ответе DNS-сервер вернул IPv4-адрес 3.125.197.172. При выполнении nslookup -type=NS запрашиваются не адреса хоста, а NS-записи, то есть имена серверов имён, обслуживающих домен.

2. Что содержится в поле Answers DNS-ответа?

Поле Answers содержит ресурсные записи того типа, который был запрошен клиентом. Для запроса A в Answers находится IPv4-адрес nginx.org: 3.125.197.172. Для запроса NS в Answers находятся имена серверов ns1.f5clouddns.com и ns2.f5clouddns.com.

3. Каковы имена серверов, возвращающих authoritative-отклик?

Для домена nginx.org в ответе на запрос NS были получены серверы имен ns1.f5clouddns.com и ns2.f5clouddns.com. Именно эти серверы являются ответственными DNS-серверами для зоны nginx.org. При этом ответ был получен от локального DNS-сервера 192.168.0.1 и в Wireshark отмечен как non-authoritative, потому что локальный сервер выступал рекурсивным резолвером.

4. Структура наблюдаемых пакетов

Трафик	**Порядок инкапсуляции**	**Назначение**
ping	Frame -> Ethernet II -> IPv4 -> ICMP -> Data	Проверка достижимости узла и анализ фрагментации IP-пакетов.
traceroute	Frame -> Ethernet II -> IPv4 -> ICMP	Построение маршрута за счет ICMP Echo Request с разными TTL и ICMP Time Exceeded.
HTTP	Frame -> Ethernet II -> IPv4 -> TCP -> HTTP	Передача HTTP-запросов и ответов прикладного уровня.
ARP	Frame -> Ethernet II -> ARP	Разрешение IP-адреса локального шлюза в MAC-адрес.
nslookup / DNS	Frame -> Ethernet II -> IPv4 -> UDP -> DNS	Получение DNS-записей типа A и NS для домена nginx.org.

Во всех пакетах, кроме ARP, кадр Ethernet содержит MAC-адреса устройств текущего локального сегмента, а IP-заголовок содержит адреса конечных узлов. ARP не использует IP как вложенный протокол, а служит для получения MAC-адреса по известному IPv4-адресу в локальной сети.

5. Выводы по работе:

В ходе выполнения данной лабораторной работы я познакомился с анализом реального сетевого трафика в Wireshark. Были рассмотрены пакеты, возникающие при работе ping, traceroute, HTTP, ARP и nslookup.

На практике удалось увидеть фрагментацию IP-пакетов, изменение TTL при построении маршрута, работу обычных и условных HTTP-запросов, а также различие между IP- и MAC-адресацией. Отдельно был рассмотрен DNS-трафик, создаваемый утилитой nslookup.

В результате стало понятнее, как данные проходят через разные уровни сетевой модели и какие служебные поля используются для передачи, маршрутизации и обработки пакетов.